Experiment No: 01

Experiment Name: Demonstration of Multilevel and Hierarchical Inheritance in Java

Task 1: Grandfather → Father → Son (Multilevel Inheritance)

Objective

1. To understand the concept of **multilevel inheritance** in Java.
2. To demonstrate how a subclass can inherit properties from multiple levels of classes.
3. To create classes representing family hierarchy (Grandfather, Father, Son).
4. To show how the Son class can access methods from both parent classes.
5.

Problem Analysis (Hybrid Model)

Multilevel inheritance occurs when a class inherits from another class, which itself inherits from another class. This forms a chain of inheritance. In this program, the Grandfather class contains a method representing a company. The Father class inherits from Grandfather and adds a car-related method. The Son class inherits from Father and adds its own method. Through inheritance, the Son class can access methods from both Grandfather and Father.

Input – Process – Output

Input:
No user input required.

Process:
Classes are created with inheritance relationships. The Son class object calls methods from all inherited classes.

Output:
Messages showing that the son has access to the company, car, and his own properties.

Algorithm:

1. Create a class Grandfather with method company().
2. Create a class Father that extends Grandfather and add method car().
3. Create a class Son that extends Father and add method bike().
4. Create an object of Son class in the main method.
5. Call all methods to demonstrate inheritance.

Source Code:

```
class Grandfather {

   void company() {
      System.out.println("Grandfather owns a company.");
   }

}

class Father extends Grandfather {

   void car() {
      System.out.println("Father owns a car.");
   }

}

class Son extends Father {

   void bike() {
      System.out.println("Son owns a bike.");
   }

}

public class MultilevelInheritanceDemo {

   public static void main(String[] args) {

      Son s = new Son();
      s.company();
      s.car();
      s.bike();
   }
}
```

Output (Example)

Grandfather owns a company.
Father owns a car.
Son owns a bike.

Discussion:

This program demonstrates **multilevel inheritance**, where the Son class inherits properties from both Father and Grandfather. It shows how Java allows classes to reuse existing functionality through inheritance. This approach improves code reuse and reduces redundancy.

Experiment no :02

Task 2: Person → Doctor / Teacher / Engineer (Hierarchical Inheritance)

Task Name: Hierarchical Inheritance Demonstration

Objective

1. To understand the concept of **hierarchical inheritance** in Java.
2. To demonstrate how multiple subclasses can inherit from a single superclass.
3. To implement a superclass Person and multiple subclasses.
4. To show how each subclass can have its own unique methods.

Problem Analysis (Hybrid Model):

Hierarchical inheritance occurs when multiple classes inherit from the same parent class. In this program, the Person class acts as the superclass containing a general method displayInfo(). The subclasses Doctor, Teacher, and Engineer inherit from Person and each defines its own method to describe their profession. This demonstrates code reuse and specialization in object-oriented programming.

Input – Process – Output

Input:
No user input required.

Process:
Superclass and subclasses are created. Objects of each subclass are created and methods are called.

Output:
Information about a person and descriptions of the professions

Algorithm:

1. Create a superclass Person with method displayInfo().
2. Create subclass Doctor extending Person with method treatPatient().
3. Create subclass Teacher extending Person with method teachStudents().
4. Create subclass Engineer extending Person with method developSystem().
5. Create objects of each subclass.
6. Call methods to display information.

Source Code:

```
class Person {
    void displayInfo() {
        System.out.println("This is a person.");
    }
}

class Doctor extends Person {
    void treatPatient() {
        System.out.println("Doctor treats patients.");
    }
}

class Teacher extends Person {
    void teachStudents() {
        System.out.println("Teacher teaches students.");
    }
}

class Engineer extends Person {
    void developSystem() {
        System.out.println("Engineer develops systems.");
    }
}
public class HierarchicalInheritanceDemo {
```

```
    public static void main(String[] args) {

        Doctor d = new Doctor();
        Teacher t = new Teacher();
        Engineer e = new Engineer();


        d.displayInfo();
        d.treatPatient();

        t.displayInfo();
        t.teachStudents();

        e.displayInfo();
        e.developSystem();
    }
}
```

Output (Example):

```
This is a person.
Doctor treats patients.

This is a person.
Teacher teaches students.

This is a person.
Engineer develops systems.
```

Discussion

This program demonstrates **hierarchical inheritance**, where multiple subclasses inherit from a single superclass. The Person class provides common functionality, while each subclass adds its own specific behavior. This approach improves code organization and promotes reusability in object-oriented programming.